NTI – Digital Design using FPGA

SPI – Master/Slave Project

Submitted by:

Radwa Ahmed Abd-Elsattar Ali

Esther George Fayez Abdo

Eyad Khaled Mabrouk Noah

Table of Contents

Abstract

This project presents the design, implementation, and verification of a configurable Serial Peripheral Interface (SPI) communication system using Verilog HDL.

The system consists of a parameterized SPI Master, an SPI Slave, and an SPI RAM module integrated through a wrapper.

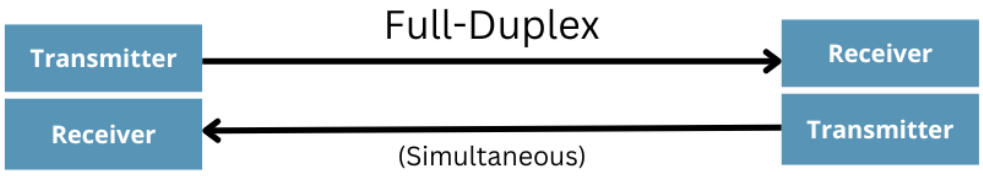
The SPI Master supports configurable data width, SPI mode selection, clock division, and selectable transmission order (MSB-first or LSB-first).

The SPI Slave receives serial commands from the master, decodes the requested operation, and communicates with the RAM to perform memory reading and writing transactions.

Functional verification was carried out through comprehensive simulation using self-checking test benches.

Introduction

The Serial Peripheral Interface (SPI) is one of the most widely used synchronous serial communication protocols in embedded systems and digital integrated circuits. It provides high-speed full-duplex communication between a master device and one or more slave devices using a small number of signal lines.



Unlike asynchronous protocols such as UART, SPI uses a dedicated clock generated by the master, allowing higher communication speeds and deterministic timing.

This project implements an SPI communication system completely in Verilog HDL suitable for FPGA implementation.

Project Objectives

- Design a configurable SPI Master.
- Design an SPI Slave capable of decoding SPI commands.
- Interface the slave with single port RAM.
- Support configurable SPI parameters.
- Verify correct data transmission using simulation.
- Produce synthesizable RTL suitable for FPGA implementation.

SPI Protocol and architecture

SPI communication uses four primary signals:

Signal	Direction	Description
SCLK	Master → Slave	Serial clock generated by the master
MOSI	Master → Slave	Master Out Slave In
MISO	Slave → Master	Master In Slave Out
CS/SS	Master → Slave	Active-low Slave Select

Communication begins when the master asserts the Chip Select (CS) line low. The master then generates clock pulses while transmitting data through MOSI and simultaneously receiving data from the slave through MISO.

The implemented SPI system consists of three major modules:

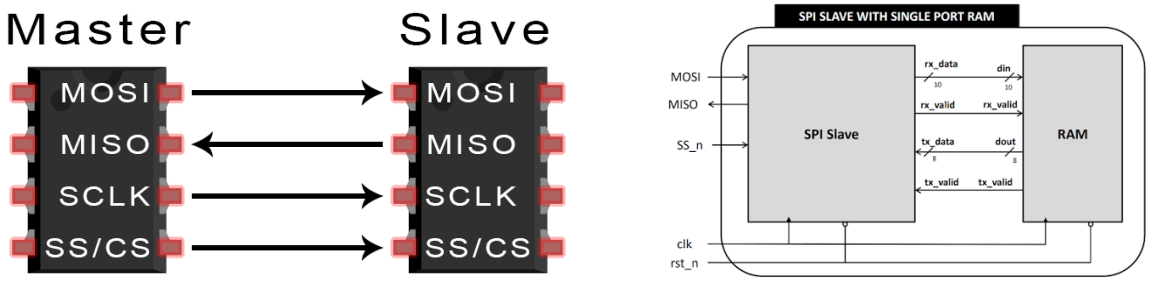


Figure 1 System Architecture of SPI

The master initiates all communication while the slave interprets incoming commands and accesses the RAM accordingly.

Methodology

SPI Master

The SPI Master was designed to be fully parameterized.

Supported parameters include:

Parameter	Description
DATA_WIDTH	Number of transmitted bits
MODE	SPI Mode (CPOL/CPHA)
CLK_DIV	Serial clock divider
MSB_FIRST	Transmission order

This allows the module to be reused in different SPI systems without modifying the RTL code.

The SPI Master operates using a four-state finite state machine.

IDLE

- Waits for the Start signal.
- CS remains high.
- SCLK remains at idle polarity.

LOAD

- Loads transmit data into the shift register.
- Clears receive register.
- Initializes counters.
- Activates Chip Select.

TRANSFER

During this state:

- Clock divider generates the SPI clock.
- SCLK toggles according to CLK_DIV.
- MOSI shifts one bit every transmission edge.
- MISO is sampled at every reception edge.
- Bit counter tracks transmitted bits.

Transmission continues until all bits have been transferred.

FINISH

- Stores received data.
- Returns SCLK to idle polarity.
- De-asserts CS.
- Raises Done flag.
- Returns to IDLE.

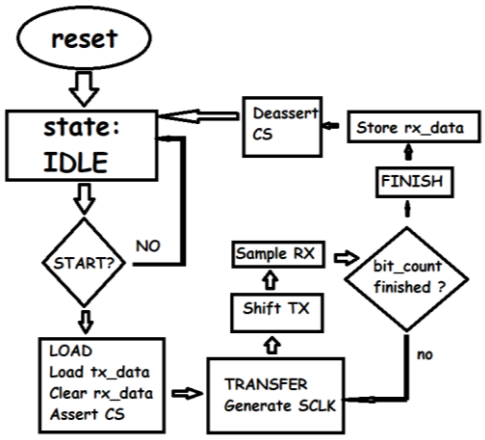


Figure 2 Flow chart of Master FSM

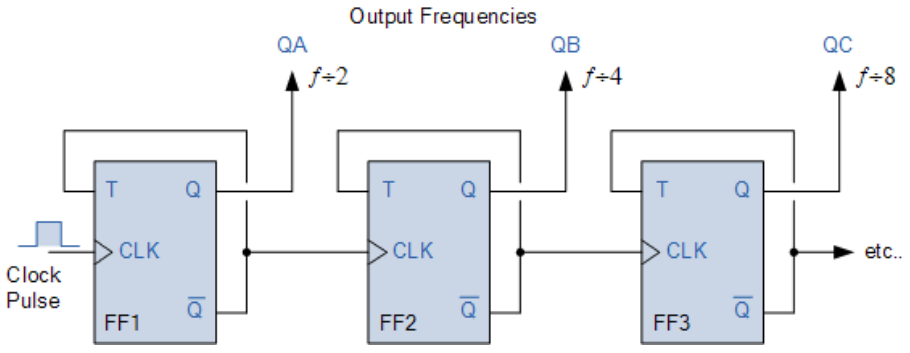
The SPI clock is generated internally from the FPGA system clock.

Instead of toggling SCLK every system clock cycle, a counter count from 0 → CLK_DIV-1 when the counter reaches CLK_DIV-1 the counter resets and SCLK toggles.

SPI Clock Frequency:

$$F\_SPI = F\_CLK / (2 \times CLK\_DIV)$$

The divider allows communication with slower slave devices while keeping the FPGA operating at a much higher frequency.



SPI Slave

The SPI Slave receives serial data from the master and interprets incoming packets.

The slave consists of two major parts:

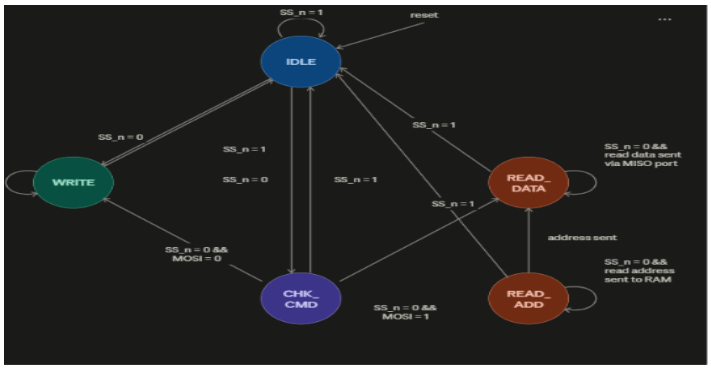
- SPI Receiver
- RAM Interface

The slave continuously monitors the Chip Select line.

Once selected, it samples incoming MOSI data using the serial clock provided by the master.

The implemented slave includes the following states:

State	Function
IDLE	Wait for CS assertion
CHECK	Decode transaction type
WRITE	Receive write command
READ_ADDR	Receive read address
READ_DATA	Transmit requested data



The slave reconstructs serial data into a parallel packet before forwarding it to RAM.

The RAM module stores user data. Supported operations include:

Command	Function
00	Write Address
01	Write Data
10	Read Address
11	Read Data

```
    else begin
        tx_valid <= 0;
        if (rx_valid) begin
            case (din [data_width+1:data_width])
            2'b00: write_adr <= din [data_width-1:0];
            2'b01: begin
                mem_array [write_adr] <= din [data_width-1:0];
            end
            2'b10: read_adr <= din [data_width-1:0];
            2'b11: begin
                dout <= mem_array [read_adr];
                tx_valid <= 1;
            end
            endcase
        end
    end
end
endmodule
```

The RAM stores 8-bit data across **256** memory locations.

This packet structure simplifies communication between the SPI slave and RAM.

Integration

After implementation of master and slave modules, integration part starts. This step is implemented after checking the functionality of the master and slave modules (will be mentioned in results section) to make sure every module works alone correctly, and to reduce possibilities of the reason for errors in the final testbench.

The integration flow focuses on controlling the master signals of transmitting and monitor the interaction between master, slave and RAM through the interconnection between them without forcing their values.

Steps followed:

1. Connection of common ports, internal signals, and check the consistency of the named parameters.

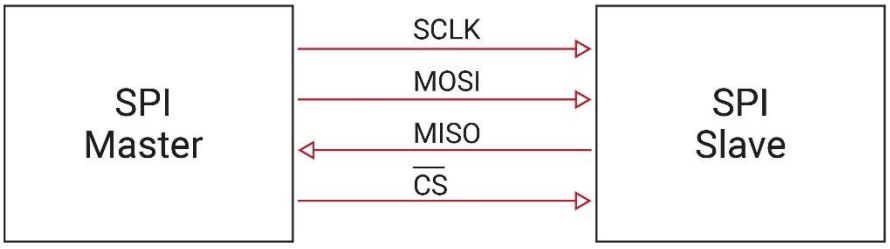


Figure 3 SPI integration

2. Adding testbench including the four operations
3. Trace the data flow between master and slave to apply synchronous timing between them.
4. Trace the data flow between slave and RAM to be able to do the four operations appropriately.
5. Omitting un-necessary and harmful resetting of data

Results and analysis

SPI Master Test bench and synthesis

Functional verification was performed using a self-checking Verilog test bench.

The following test cases were evaluated:

- Reset functionality
- Multiple transmit patterns
- All zeros
- All ones
- Alternating bits
- Random data
- Variable DATA_WIDTH
- Variable CLK_DIV
- Clock edge verification

Loopback testing connected MOSI → MISO allowing transmitted data to be immediately received and compared automatically.

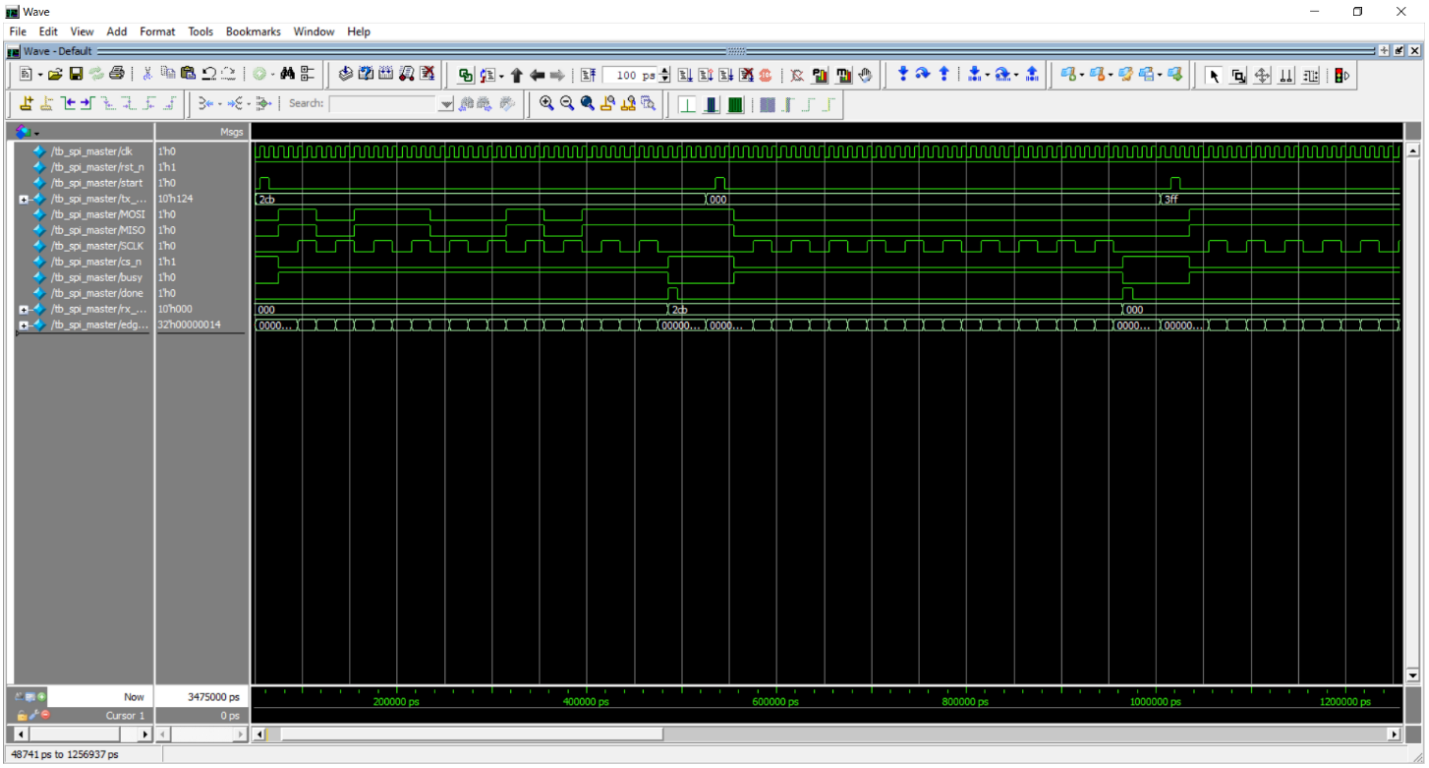


Figure 4 Master test bench

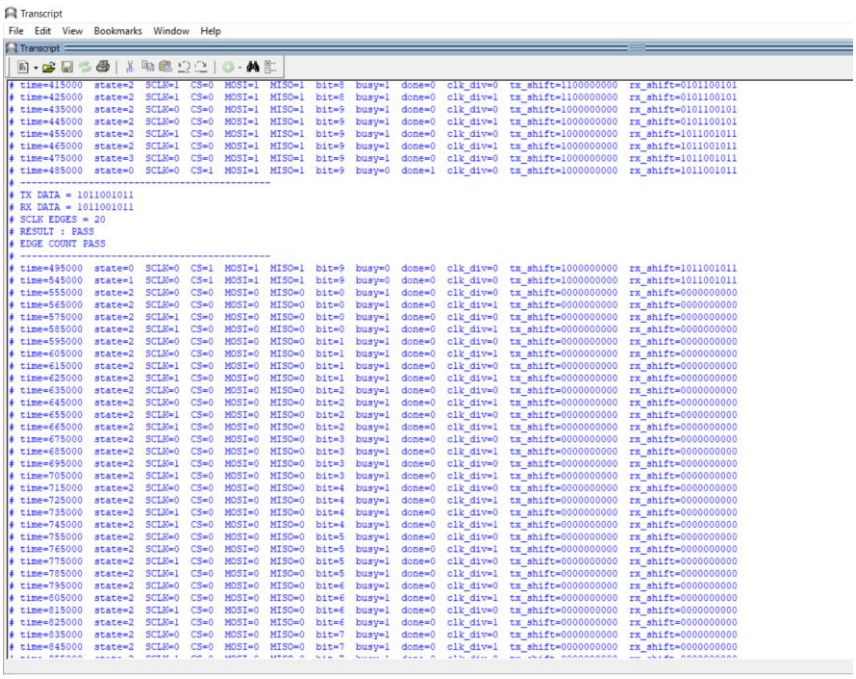


Figure 5 Master Test bench

During synthesis, the RTL description was translated into FPGA resources such as Look-Up Tables (LUTs), Flip-Flops (FFs), multiplexers, and clock routing resources.

The synthesis tool inferred the following hardware components from the RTL design:

- Finite State Machine (FSM) controlling the SPI transaction.
- Transmit shift register for serializing parallel input data.
- Receive shift register for capturing serial input data.
- Bit counter is used to determine the completion of data transmission.
- Programmable clock divider used to generate the SPI serial clock (SCLK).
- Output control logic for the **MOSI**, **SCLK**, **CS**, **busy**, and **done** signals.

Logic optimization was performed automatically by the synthesis tool while preserving the intended functionality of the design. Since the SPI Master is fully parameterized, the synthesized hardware can be configured for different data widths, clock division ratios, transmission order (MSB-first or LSB-first), and SPI operating modes without modifying the RTL code.

The generated RTL schematic verifies the architecture of the SPI Master and confirms the correct interconnection between the control FSM, clock divider, shift registers, counters, and output control logic.

Following synthesis, timing analysis was performed to verify that the synthesized logic satisfies the required timing constraints before placement and routing.

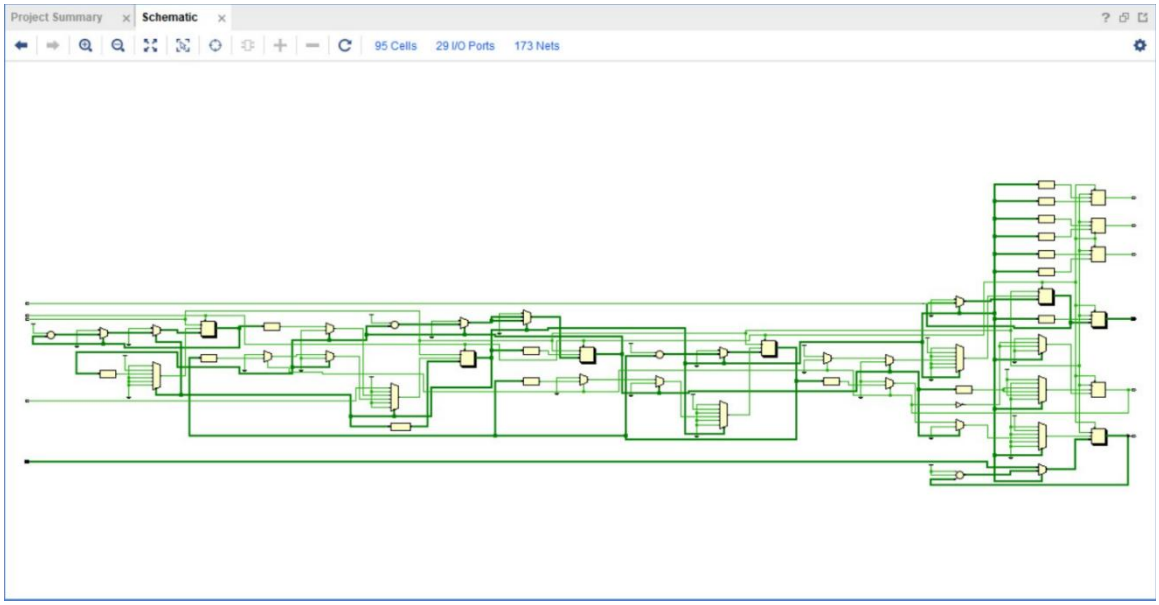


Figure 6 Master RTL schematic

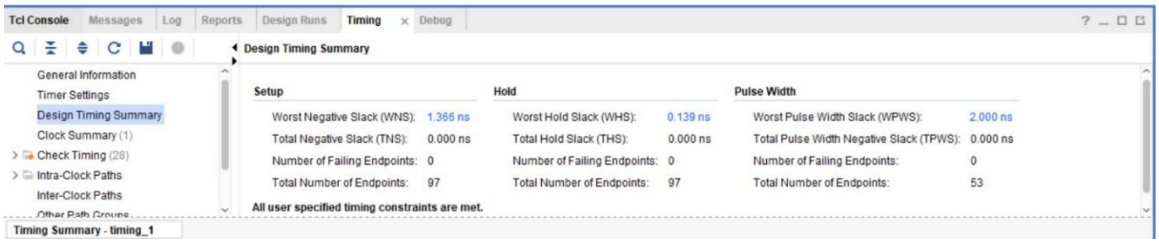


Figure 7 Before PnR timing analysis for Master

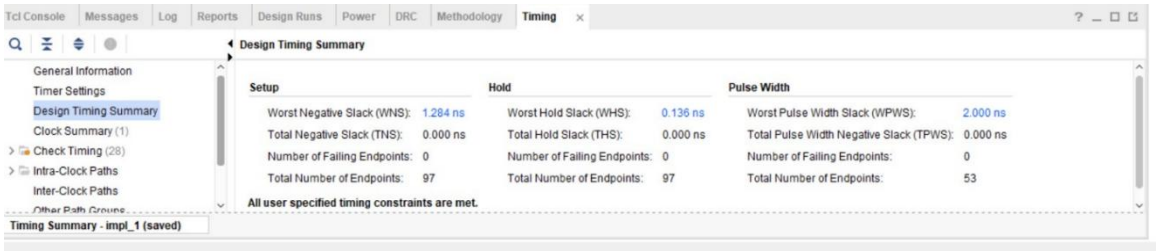


Figure 9 After PnR timing analysis for Master

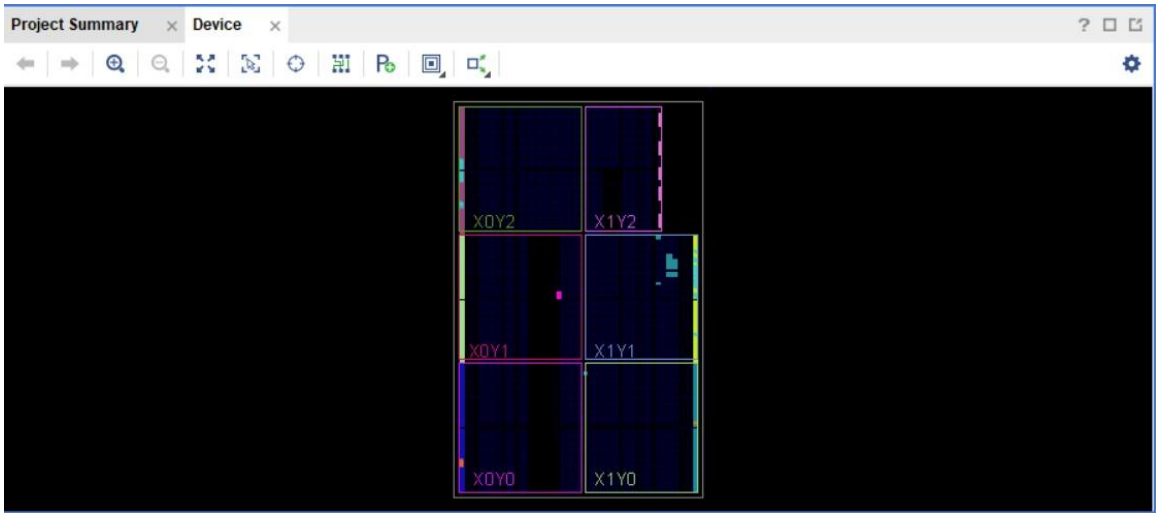


Figure 10 Utilized sections of FPGA

SPI Slave Test bench and synthesis

The verification environment automatically:

- Generates reset sequence.
- Starts SPI transactions.
- Counts SCLK edges.
- Checks received data.
- Reports PASS/FAIL automatically.
- Monitors internal FSM states.

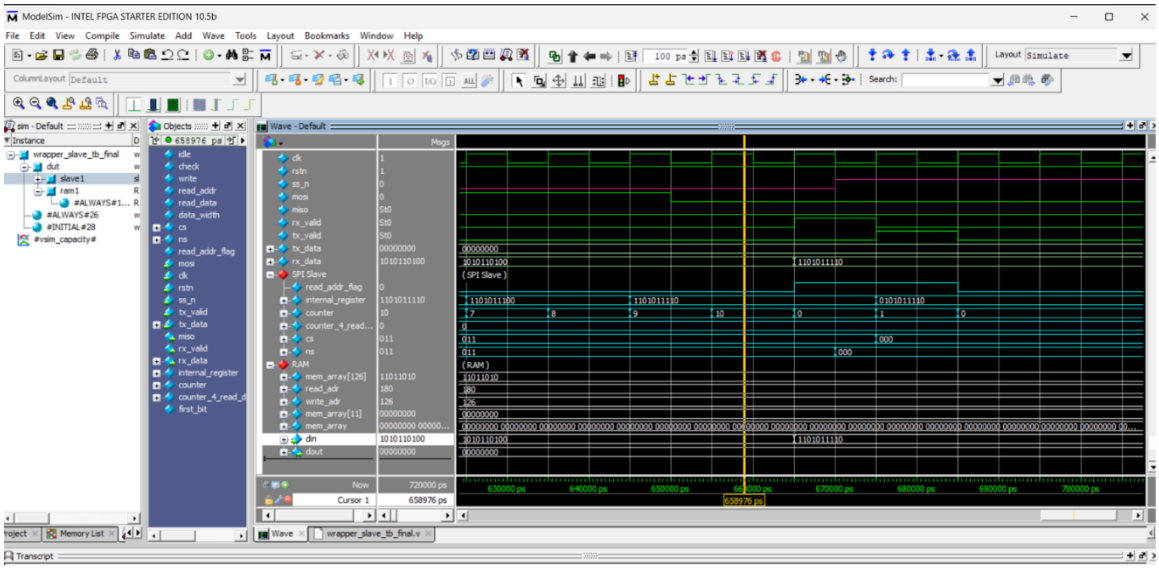


Figure 11 Read data for slave teset bench

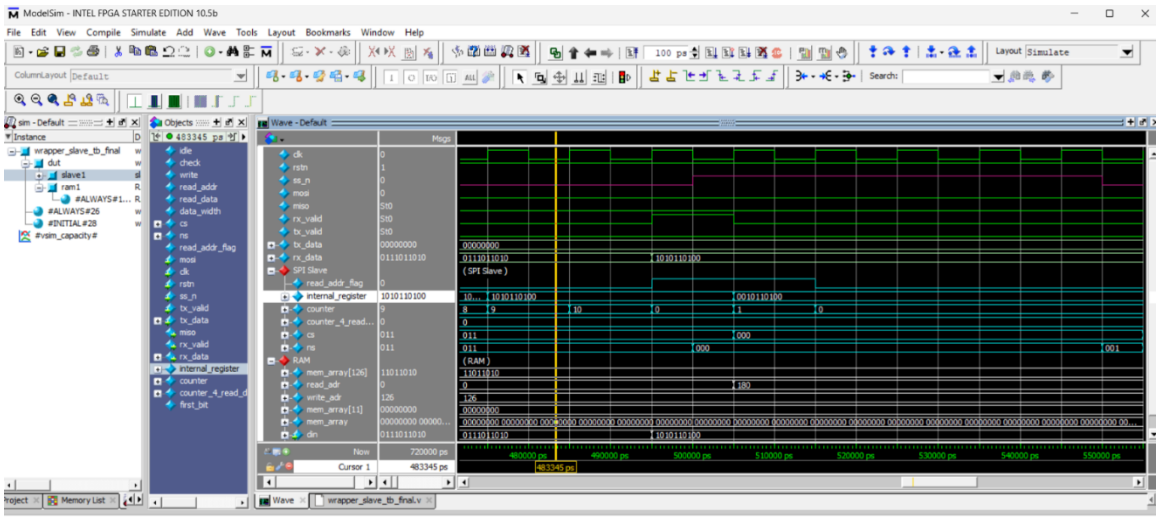
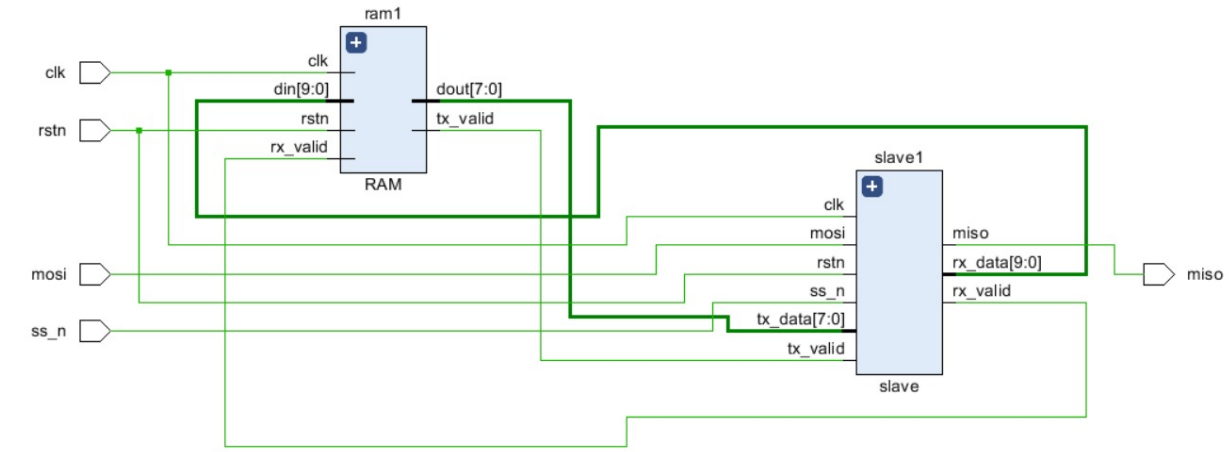


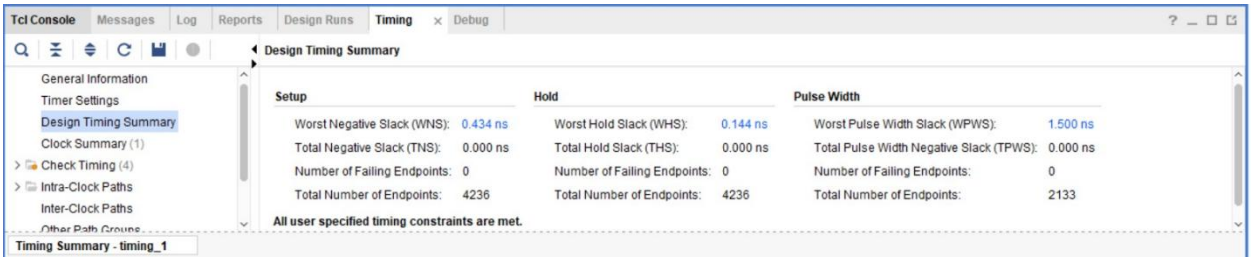
Figure 12 Read address for slave test bench

The synthesis process converted the behavioral Verilog description into FPGA logic resources while preserving the communication protocol required by the SPI interface.

- The synthesized hardware consists of the following major functional blocks:
- Slave Finite State Machine (FSM).
- Serial receiving logic for collecting incoming MOSI data.
- Serial transmitting logic for driving the MISO line.
- Receive data register.
- Bit counters controlling serial reception and transmission.
- Interface logic connecting the slave controller with the RAM module.
- The synthesis tool optimized the control logic and datapath while maintaining correct synchronization between the SPI signals and the internal memory interface. The synthesized RTL schematic confirms the implementation of the slave controller and the proper connection of its receive, transmit, and state-control circuitry.
- Static timing analysis performed after synthesis verified that the synthesized design satisfies the specified timing requirements prior to physical implementation.



Block diagram of slave & RAM



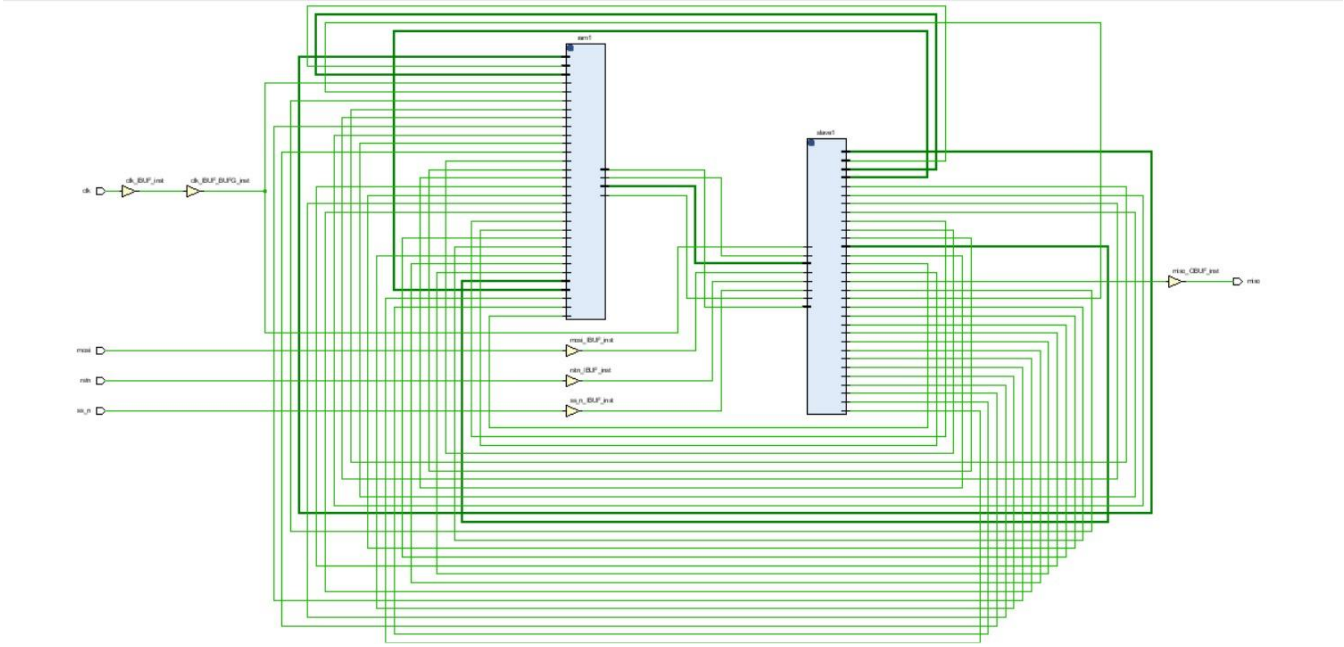
Timing analysis Pre-PnR (fitting)

Design Timing Summary

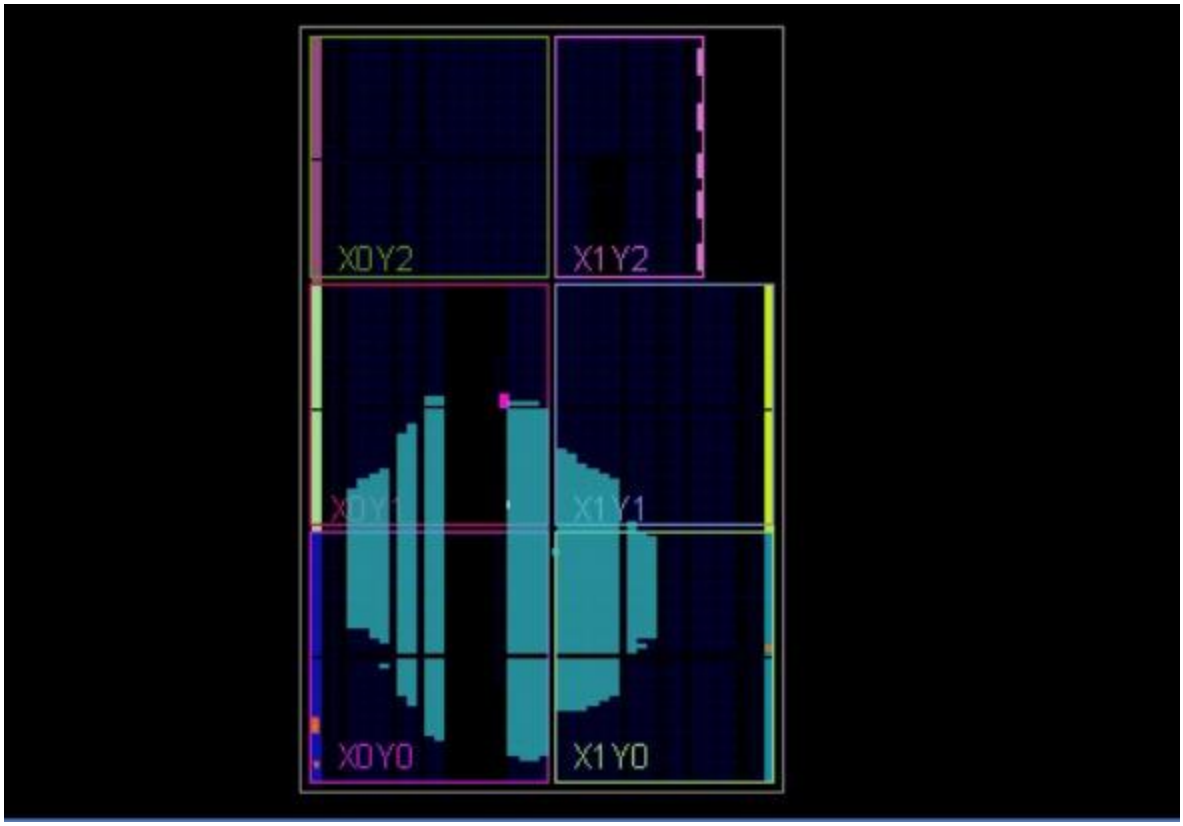
Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	0.118 ns	Worst Hold Slack (WHS):	0.056 ns	Worst Pulse Width Slack (WPWS):	2.000 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	4236	Total Number of Endpoints:	4236	Total Number of Endpoints:	2133

All user specified timing constraints are met.

Timing analysis after PnR



Implementation, placement & routing of slave and RAM



Placement & routing of spi master in FPGA (Utilized sections of FPGA)

SPI integration Test bench and synthesis

The test bench results show the productive flow of steps of integration for correcting timing between the two modules.

Data is successfully transferred from master to slave through MOSI.

Slave was successfully adjusted to be synchronized with the data on MOSI and contact with the associated RAM.

Finally, the slave was adjusted to be able to return the data-when calling its operation-back again to the master.

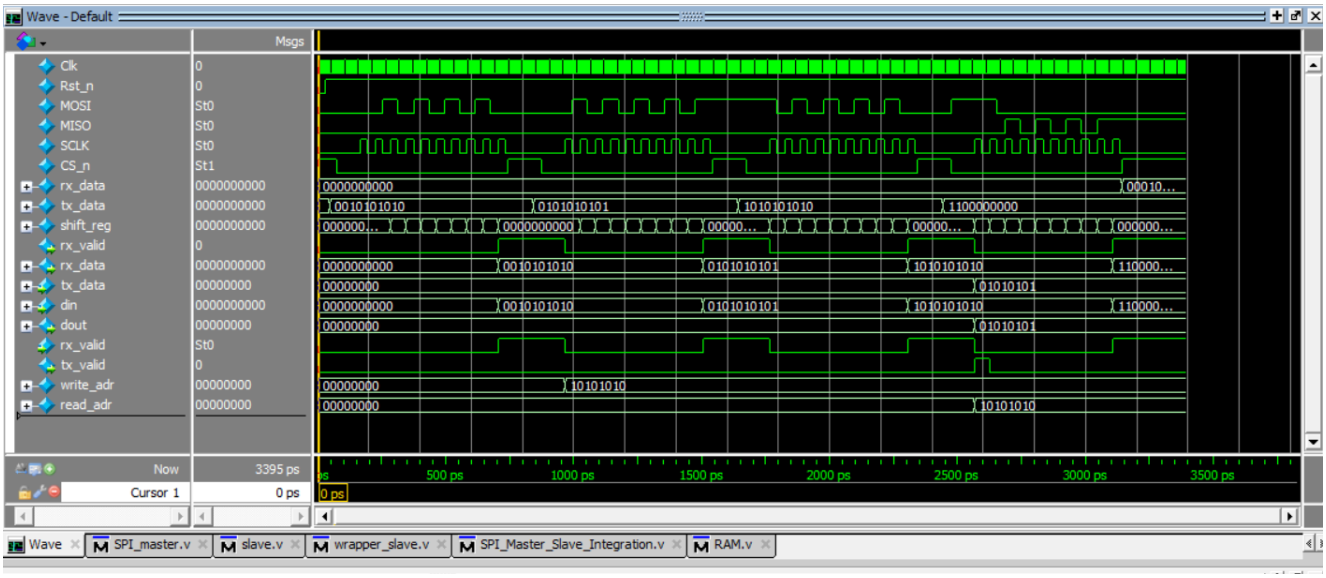


Figure 9 Integration test bench

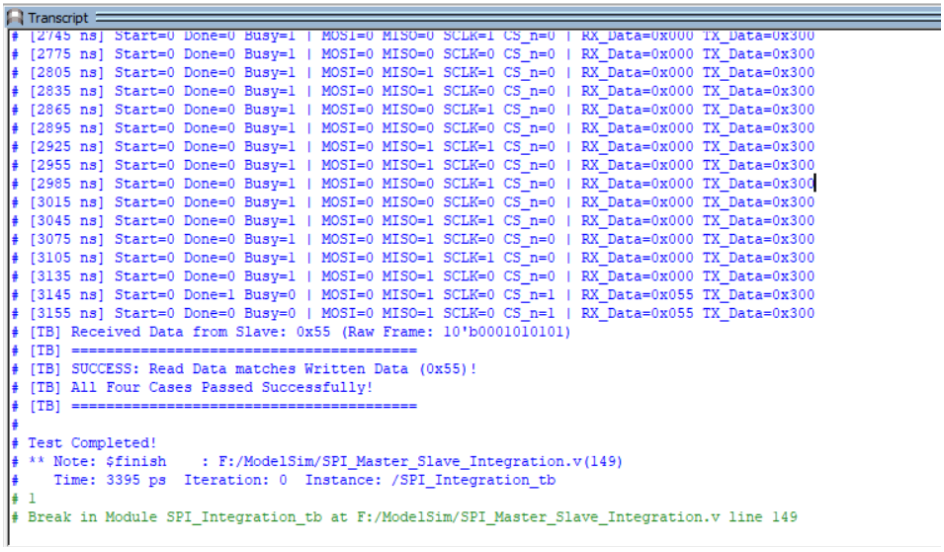


Figure 10 Integration results on console

Conclusion

A complete SPI communication system was successfully designed and verified using Verilog HDL. The project demonstrated the implementation of a configurable SPI Master, an SPI Slave, and an integrated RAM interface capable of executing serial read and write operations. Through parameterization and modular RTL design, the architecture can be adapted to different communication requirements and FPGA platforms.

Comprehensive simulation verified correct clock generation, serial data transmission, packet decoding, and memory access, demonstrating the functionality and reliability of the proposed design. The project also provides a scalable foundation for implementing more advanced SPI features such as support for all SPI modes, multiple slave devices, burst transfers, and FIFO-based buffering in future developments